

Using Class and Domain Information to Address Domain Shift in Federated Learning

Chien-Yu Chiou¹, Member, IEEE, Chun-Rong Huang², Senior Member, IEEE, Lawrence L. Latour³,
Yang C. Fann⁴, and Pau-Choo Chung⁵, Life Fellow, IEEE

Abstract—In federated learning (FL), heterogeneous and client-specific data distributions cause a domain-shift problem, which leads to divergent local models and degraded global performance. To address this problem, this study proposes a class- and domain-aware FL framework that decouples and collaboratively learns the domain-invariant and domain-specific representations. During client training, a novel cross-gated feature separation (CGFS) module is employed to separate the domain features from the class features. A heterogeneous prototype contrastive learning (HPCL) module is then used to guide the learning of the class features and domain features with good discriminability within each feature space. Finally, during server aggregation, a gradient-reweighted hierarchical aggregation (GHA) strategy is applied to effectively aggregate information from all the clients and build a global model with good robustness to domain variation. The experimental results obtained on two FL datasets with domain shift show that the proposed method consistently outperforms state-of-the-art approaches.

Index Terms—Contrastive learning, domain shift, federated learning (FL), prototypes.

I. INTRODUCTION

FEDERATED learning (FL) [1] is a promising strategy for preserving data privacy and represents a major paradigm shift in the artificial intelligence domain. The FL architecture typically consists of a server and multiple clients, and the training is conducted through multiple

communication rounds that alternate between client training and server aggregation. In each communication round, the participating clients train local models using their own private data and then upload these models to the server. The server then aggregates these models to generate a global model. The global model is typically built through the weighted summation of the local models, in an approach equivalent to the summation of local gradients [1]. The global model is then dispatched to the clients for FL in the next round. In settings where the data distributions of the clients are independent and identically distributed (i.i.d.), the global model shows a comparable performance to that of normal deep learning models with access to all the data [1]. However, clients often have heterogeneous (non-i.i.d.) data distributions [2], which can degrade the performance of the model.

This article addresses a particularly challenging form of heterogeneous data distribution called the domain-shift problem [3], [4], [5], [6]. Fig. 1(a) illustrates a typical example of the considered problem. Client 1 captures images of bicycles and laptops using normal cameras in various uncontrolled environments. Consequently, the collected images have only moderate clarity and resolution and large variations in lighting, shading, and background. In contrast, Client 2 collects images using high-end digital single-lens reflex cameras in controlled environments. Thus, the collected images have high resolutions with clean backgrounds and minimal variations in lighting and shading. Although the class is the same for both clients, the visual appearance of the objects in the images is strongly shaped by the data collection context. Therefore, the local datasets encode not only the class information but also the domain information. When local models are trained independently on such divergent domains, the aggregated global model struggles to align these representations. This leads to significant performance degradation, the core of the domain-shift problem in FL.

Moreover, under domain shift, the learned features inevitably entangle class- and domain-relevant information. For example, as shown in Fig. 1(a), the learned features belonging to the same class but coming from different domains are separated, while the features belonging to different classes but in different domains overlap. In this case, the features represent the domains rather than the classes, making it difficult for the model to discriminate among the different objects. In addition, each client fits its local model using its own private data during client training. As a result, the model gradients from different clients diverge significantly and convey highly domain-dependent learning signals. If the server then aggregates these gradients weighted purely by the volume

Received 15 May 2025; revised 2 November 2025 and 30 November 2025; accepted 19 January 2026. Date of publication 6 February 2026; date of current version 2 September 2026. This work was supported in part by the National Science and Technology Council of Taiwan under Grant NSTC 114-2221-E-A49-034-MY3, Grant NSTC 114-2634-F-006-002, Grant NSTC 114-2640-E-006-010, and Grant NSTC 112-2926-I-006-504-G; and in part by the National Institutes of Health's Intramural Research Program at the National Institute of Neurological Disorders and Stroke. (Corresponding authors: Chun-Rong Huang; Pau-Choo Chung.)

Chien-Yu Chiou is with the Department of Electrical Engineering, National Cheng Kung University, Tainan 701, Taiwan (e-mail: n28104066@gs.ncku.edu.tw).

Chun-Rong Huang is with the Department of Computer Science and the Institute of Multimedia Engineering, National Yang Ming Chiao Tung University, Hsinchu 300, Taiwan (e-mail: crhuang@cs.nycu.edu.tw).

Lawrence L. Latour is with the Division of Intramural Research, National Institute of Neurological Disorders and Stroke, National Institutes of Health, Bethesda, MD 20814 USA (e-mail: lawrence.latour@nih.gov).

Yang C. Fann is with the Division of Intramural Research, National Institute of Neurological Disorders and Stroke, National Institutes of Health, Bethesda, MD 20892 USA (e-mail: fann@ninds.nih.gov).

Pau-Choo Chung is with the Department of Electrical Engineering, National Cheng Kung University, Tainan 701, Taiwan, and also with the Department of Electrical Engineering, National Chung Cheng University, Chiayi 621, Taiwan (e-mail: pcchung@ee.ncku.edu.tw).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3658584>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3658584

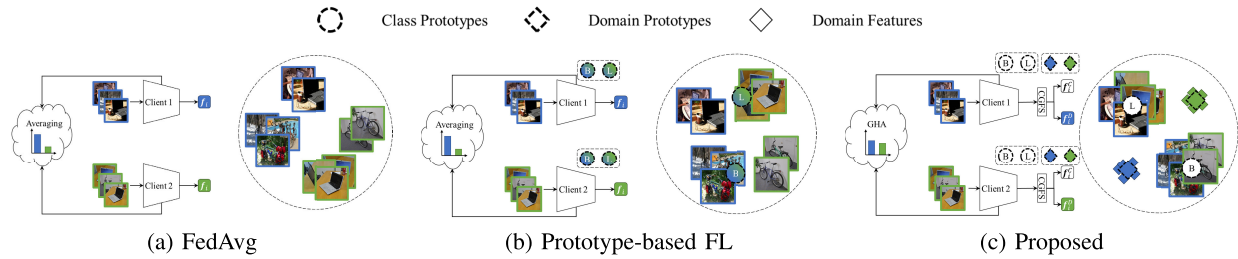


Fig. 1. Comparison of conventional FL methods and proposed method. Clients 1 and 2 contain private data of the same classes (bicycle and laptop) of different domains. (a) Image features of the same class in different domains learned by FedAvg are separated, while the features of different classes in the same domain may be mixed together. (b) By aligning features with class prototypes, the features of the same class learned by prototype-based FL methods become closer, while the distribution is still affected by the domains. (c) Proposed method separates and learns class features and domain features individually by using the proposed CGFS module, class prototypes, and domain prototypes. Thus, the servers can aggregate class features of the same class from different domains by using the proposed GHA.

of data at each client, the global model becomes biased toward the clients with dominant data volumes and may underfit those with fewer data [4], [7]. Therefore, mitigating domain shift in both feature representation learning and model aggregation is essential for improving the performance and fairness of FL.

However, in the FL setting, clients do not have access to the data of other domains. Thus, conventional domain adaptation methods [8], domain generalization methods [9], and transformation-invariant feature methods [10] that rely on centralized access to multidomain data to learn domain-invariant features cannot be directly applied.

Many prototype-based FL methods have been proposed to alleviate this problem [3], [11], [12], [13]. As shown in Fig. 1(b), these methods aim to improve the model performance by aligning features of the same class using class prototypes, where these prototypes are usually calculated by averaging the features of each client. However, existing prototype-based methods focus only on semantic class information while ignoring domain characteristics. Thus, the learned features of different clients for the same class may still be affected by the domain information. In addition, only the features are aligned with the class prototypes, while the local model parameters remain divergent across clients. As a result, the models from different clients still contain diverse domain information. Consequently, when aggregating the global model at the server, the diverse directions of the local models may still cause the model to be biased toward the majority data. Thus, despite prototype-based alignment, domain shift remains unresolved in these methods.

To solve these problems, the present study proposes a class- and domain-information-aware FL framework that explicitly separates and learns class-discriminative and domain-specific representations during client training, as shown in Fig. 1(c). The learned information is then jointly utilized to help generate a more reliable global model during server aggregation. By properly disentangling and integrating the class and domain cues, features belonging to the same class across different domains can be aligned more accurately, thereby substantially reducing the impact of the domain-shift problem.

To learn the domain information and class information from each client, a novel client-side learning network is proposed, as shown in Fig. 2. The framework consists of a cross-gated feature separation (CGFS) module and a heterogeneous prototype contrastive learning (HPCL) module. The CGFS module disentangles the features of the input images into class features and domain features. In each client, the class features

encode the semantic cues needed for object recognition, while the domain features capture the appearance variations caused by the client-specific imaging conditions (e.g., resolution, background, and illumination). During server aggregation, the class features and domain features of the different clients are aggregated to generate separate class prototypes and domain prototypes, respectively. These prototypes are then transmitted back to each client to help the HPCL module learn consistent class features and domain features in subsequent rounds of the FL process.

To drive client training, a feature separation loss is first applied to distinguish the class features from the domain features. To enhance the discriminability of the separated features, a class contrastive loss and class classification loss are then used to reinforce the semantic clustering of the class features. Similarly, a domain contrastive loss and a domain classification loss are utilized to learn the representative domain features. Finally, a continual regularization loss is applied to stabilize the local model updates and mitigate overfitting to private client data. After client training, the class features and domain features in each client are clustered to represent the feature distribution of the client. The local models and clustered features are then uploaded to the server, as shown in Fig. 2.

To prevent the global model from being biased toward the clients with dominant data volumes, this study further proposes a gradient-reweighted hierarchical aggregation (GHA) method on the server side. First, the local client models are clustered in parameter space to identify groups with similar domain characteristics. The server then aggregates models within the same domain cluster to produce domain-specific models, which are then combined to form a global model. To effectively aggregate the local models, a gradient-aware model aggregation (GMA) strategy is employed to adjust the summation weights based on both the gradients of the local models and their relative data volumes. After model aggregation, the global class prototypes and global domain prototypes are calculated using the clustered feature distributions of all the clients. Finally, the global model and global prototypes are dispatched to the clients for FL training in the next round. By jointly optimizing the client-side representation learning and server-side aggregation processes, the proposed framework more effectively integrates the heterogeneous domain information, thus improving the overall performance of the FL model.

The contributions of this study can be summarized as follows.

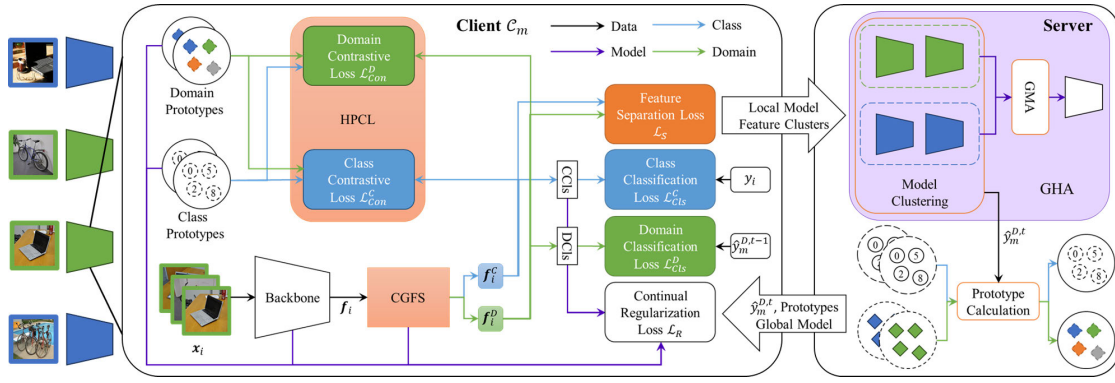


Fig. 2. Overview of the proposed method. In each client, the backbone extracts the features f_i of the image x_i . The proposed CGFS module learns to separate the image features f_i into class features f_i^C and domain features f_i^D using the feature separation loss $\mathcal{L}_S$. Then, the proposed HPCL module guides feature learning by using the class contrastive loss $\mathcal{L}_{Con}^C$ and domain contrastive loss $\mathcal{L}_{Con}^D$ with the class prototypes and domain prototypes. The class classifiers (CCls) and domain classifiers (DCls) learn to identify the class and domain by using the class classification loss $\mathcal{L}_{Cls}^C$ and domain classification loss $\mathcal{L}_{Cls}^D$ based on the class label y_i and the server-assigned domain label $\hat{y}_m^{D,t-1}$, respectively. The continual regularization loss $\mathcal{L}_R$ constrains the model update direction. After client training, the local models and feature clusters of each client are uploaded to the server to build the global model. The server first clusters the client models and assigns domain labels $\hat{y}_m^{D,t}$ based on the parameters of the local models. Then, the global model is built by using the proposed GMA method and local model clusters. After model aggregation, the global class prototypes and global domain prototypes are calculated based on $\hat{y}_m^{D,t}$ and the feature clusters from the clients.

- 1) First, to address the domain-shift problem in FL, a novel CGFS module and HPCL module are proposed to separate and learn the domain-specific features and class-discriminative features independently.
- 2) Second, a GHA method is introduced on the server side to more effectively integrate information from clients with different domains.
- 3) Third, extensive experiments on two FL datasets with domain-shift settings confirm the superiority of the proposed method compared with state-of-the-art methods.

The remainder of this article is organized as follows. Section II discusses related research on FL with heterogeneous client data. Section III introduces the proposed framework in detail. Section IV presents and discusses the experimental results. Finally, Section V provides some brief conclusions.

II. RELATED WORK

A. FL for Heterogeneous Data

To address the problem of heterogeneous data distribution in FL, several methods have proposed constraining the update directions of the client models to enforce greater consistency among them [7], [14], [15], [16], [17], [18], [19], [20], [21]. However, these techniques generally rely on the previously aggregated global model to regularize the client updates. Overly constraining the local models may limit their ability to learn new data, while insufficient constraints may fail to prevent divergence across clients, leading to persistent heterogeneity in the learned models.

Some methods attempt to improve the integration of the information received from different clients during server aggregation by adjusting the summation weights or clipping part of the gradients [4], [22], [23], [24]. For example, the summation weights of the local models may be adjusted based on criteria such as their performance [23], loss [24], or temporal gradient consistency [4]. Li et al. [25] found that global models with smaller norms show better generalization ability and accordingly introduced a learnable global shrinking factor to improve the generalization ability of the global

model. Shi et al. [26] proposed a layer-wise adjustment of the global shrinking factor to further improve the performance. To improve model consensus, FedAWA [27] adaptively increases the aggregation weights of the local models closer to the current global model in the parameter space.

Farhadkhani et al. [28] applied static clipping to the local model gradients to improve the robustness of the model toward false gradients sent by adversarial clients. In a more recent study, Allouah et al. [29] proposed an adaptive clipping method to further improve the model robustness. However, in the absence of additional constraints during client training, the updated gradients of the different local models are still diverse due to the domain-shift problem. The diverse gradients conflict with each other and lead to less effective aggregation at the server when constructing the global model.

Another line of research leverages knowledge of the data distributions of the other clients during local training. To preserve data privacy, this information is typically transferred through surrogate representations, such as heuristic mathematical models specific to the particular problem [30], feature decomposition matrices [31], adaptive augmentation policies [32], learned offset models [33], or generative models [34], [35]. Other approaches generate augmentation samples to simulate the data distributions of the other clients. For example, Zhou et al. [5] generated local training samples based on the statistics of the intermediate layer feature distributions of the other clients. Similarly, FedRDN [36] randomly mixes the mean and standard deviation between clients to diversify the training space. By transferring data distributions, these methods help clients exploit the data distributions of the other clients to enhance their training performance. However, heuristic models may not fully represent the nonlocal data distributions. Furthermore, the need for offset models and generative models increases the computational cost and remains susceptible to domain shift, particularly when the client distributions differ substantially.

Some recent studies have built multiple global models to specifically fit the data of each domain or client rather than learning a single universal model. For instance,

in [37], [38], and [39], the server clusters the clients into multiple domains and builds a cluster model for each cluster based on the local models within the same cluster. However, this approach requires the server to maintain multiple models rather than just one. Furthermore, the cluster models learn little information from the clients in different clusters, thereby limiting the ability of the model to generalize across domains. In a different approach, studies [40], [41], [42], [43], [44], [45] build additional personal adaptive models in clients to personalize the shared global model to the private data of each client. In a recent study [6], domain-specific skew erasers were proposed to suppress domain-specific features and enhance feature consensus. Although personalization-based methods improve adaptation to private client data, they require the clients to train and maintain additional personal models, and the global model may still be impaired by the domain-shift problem.

The personalized FL process can be formulated as a meta-learning procedure that aims to learn a model capable of rapid adaptation to different tasks [46]. In such methods, the server builds a global metamodel using the information provided by the participating clients, and each client then personalizes this model based on its own private data. AFM3D [47] applies a first-order approximation to the client model gradients to reduce the computational cost of meta learning. Similarly, Wang et al. [48] limit meta learning to only the participating clients and predict the updates of the nonparticipating clients using an attention-based model. You et al. [49] proposed a framework for personalizing Internet of Things (IoT) models using federated meta learning. Unlike centralized meta-learning, where full access to multitask data and gradients is available, federated meta-learning approaches require client updates to be performed locally and model aggregation to occur with limited information sharing. This leads to higher system complexity and substantial challenges in efficiently training models across distributed devices. This, in turn, may adversely affect the convergence and overall performance of the model [46].

Since FL clients have no access to the data of other clients, catastrophic forgetting may occur during local training, causing the client models to forget knowledge learned in previous rounds [19]. To solve this problem, federated incremental learning methods have been proposed that incrementally learn new unseen tasks while retaining the knowledge acquired from previous tasks. For example, Li et al. [50] trained a set of personalized local models at each client, with each model corresponding to at least one domain task. The final predictions were then aggregated based on a knowledge-matching score across tasks. Zhang et al. [51] assigned unique orthogonal frequency components to different clients and learned personalized local models independently to avoid interference that might induce catastrophic forgetting. ReFed [52] built a personalized informative model at each client to estimate the importance scores of the training samples and then selected replay samples based on both importance and global class distribution statistics.

Since clients cannot share cached data, cache-based federated incremental learning methods are effective only when the clients encounter new unseen tasks. Furthermore, personalization-based methods require the clients to train

and maintain additional personal models, which increases the computation and storage costs at the client.

B. Prototype-Based FL

More recently, prototype-based methods have been proposed that employ prototypes as anchors for class-discriminative features during learning. In particular, clients align their learned features to server-provided prototypes throughout training. Some methods predefine the prototypes using handcrafted formulations or pretrained models [53], [54]. For example, Shi et al. [54] adopted a pretrained CLIP model [55] to define class prototypes. However, predefined prototypes may not accurately represent the real feature distributions. Moreover, they are not guaranteed to fit the evolving data at the client during training. Consequently, other methods compute the prototypes dynamically based on the features of each class provided by the participating clients [3], [11], [12], [13], [43], [56], [57], [58], [59], [60], [61], [62]. FedGMKD [58] builds representative prototypes and soft predictions of each class using a Gaussian mixture model (GMM) and then adjusts the model aggregation weights based on the quality and quantity of the prototypes and soft predictions. An et al. [62] further improved feature alignment by leveraging correlations among semantically related classes.

To further enhance the discriminability among different classes, several prototype-based contrastive learning methods have been proposed [3], [56], [63]. For example, Huang et al. [3] increased the interclass discriminability by performing contrastive learning using cluster prototypes and enhanced the feature consistency within the same class using unbiased prototypes. Wang et al. [12] proposed an α -sparsity prototype loss to accommodate the distribution sparsity of interclass features. To further enhance the relationship between the features and prototypes, pFedCSPC [43] regularized the relative angles and distances between the features and prototypes of different classes. FedSA [60] employs predefined class anchors and margin-enhanced contrastive learning to enforce class boundaries at the client side and updates these anchors at the server based on the exponential moving average of the client feature statistics. FedMPS [64] proposes to build multilevel prototypes to capture high-level semantic features and detailed low-level features.

Rather than attempting to guide the client feature training process, some methods focus on generating well-separated prototypes on the server side. FedTGP [13] trains a global prototype generator on the server based on the client features using a contrastive learning approach with an adaptive interclass margin. FedLSA [59] enhances server-side prototype separability by applying a similarity-based separation loss, supported by a global semantic-aware classifier.

Feature-based prototypes provide information about the underlying feature distribution and therefore improve feature learning. However, they typically consider only the class information. Consequently, the local models and features learned by different clients may still retain domain-specific variations and remain diverse, leading to the domain-shift problem. Moreover, the global model and prototypes generated through simple averaging of the diverse local models and features may be biased toward clients with dominant data volumes.

To address the domain-shift problem, the method proposed in this study learns the class information and the domain information individually. In particular, the CGFS module first disentangles the domain features from the class features at the client side. The HPCL module then enforces consistency among the class features belonging to the same semantic category, thereby reducing divergence among the client models during local training. Finally, the heterogeneous local models are effectively aggregated at the server side using the GHA strategy based on the domain characteristics and gradient behavior. Overall, through joint optimization on both the client side and the server side, the proposed framework mitigates the domain-shift problem and thus improves performance on FL tasks involving heterogeneous data distributions.

III. METHOD

A. Problem Definition

Let $\mathcal{S}$ represent the server in FL and $\{\mathcal{C}_m\}_{m=1}^M$ represent a set of M participating clients. Each client has its own private data $\mathcal{D}_m = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_m}$, where $\mathbf{x}_i$ and y_i are the i th image and its class label, and N_m is the total number of images. The aim of the FL process is to train a global model $F(\cdot)$ with parameters θ that fit the data of all the clients through T successive training rounds. Each round involves a client training stage and a server aggregation stage. During the client training stage of the t th round, the m th client trains a local model $F_m^t(\cdot)$ with parameters θ_m^t to minimize the training loss $\mathcal{L}_m^t$ in the private data $\mathcal{D}_m$ as follows:

$$\theta_m^* = \arg \min_{\theta_m^t} \sum_{\mathbf{x}_i, y_i \in \mathcal{D}_m} \mathcal{L}_m^t(F_m^t(\mathbf{x}_i; \theta_m^t), y_i). \quad (1)$$

After client training, the local models are sent to the server to build a global model with parameters θ^t via the following weighted summation:

$$\theta^t = \sum_{m=1}^M w_m \theta_m^*, \quad \text{s.t.} \quad \sum_{m=1}^M w_m = 1 \quad (2)$$

where w_m is the summation weight of the m th local model.

Since the data collected from different clients usually have different characteristics, the data distributions $P_m(\mathbf{x}|y)$ and $P_n(\mathbf{x}|y)$ of two different clients may be different for the same class y , as formulated in the following:

$$P_m(\mathbf{x}|y) \neq P_n(\mathbf{x}|y), \quad \text{s.t.} \quad P_m(y) = P_n(y). \quad (3)$$

These different distributions imply that the data contain not only class information but also domain information. Local models trained on data with different distributions will inevitably diverge, leading to the well-known domain-shift problem [3], [4], [5], [6]. This divergence poses a significant challenge, as the server must aggregate these heterogeneous local models into a single global model capable of representing the data from all the clients, regardless of the domain differences between them.

B. Overview

To solve the domain-shift problem, the FL framework proposed in the present study learns the class information and the domain information separately during the client training stage and then hierarchically aggregates the local models of

the clients from different domains using a gradient-aware aggregation strategy, as shown in Fig. 2. During training at client $\mathcal{C}_m$, the proposed CGFS module learns to disentangle the class features and domain features of the local images. The proposed HPCL module then guides feature learning through the use of dual-class prototypes $\mathcal{P}_m^t = \{\mathcal{P}_m^{1,t}, \mathcal{P}_m^{2,t}\}$ and dual-domain prototypes $\mathcal{Q}_m^t = \{\mathcal{Q}_m^{1,t}, \mathcal{Q}_m^{2,t}\}$ provided by the server. The first group prototypes, $\mathcal{P}_m^{1,t}$ and $\mathcal{Q}_m^{1,t}$, are adaptively learned in round t based on private data during client training, while the second group prototypes, $\mathcal{P}_m^{2,t}$ and $\mathcal{Q}_m^{2,t}$, are provided by the server in round t to serve as fixed anchors. Based on $\mathcal{P}_m^t$ and $\mathcal{Q}_m^t$, the HPCL module learns the representative class features and domain features of the private data during client training, and these learned features are then clustered to represent the class and domain feature distributions of the client. Finally, the local models and clustered features are uploaded to the server for aggregation.

During server aggregation, the proposed GHA strategy is employed to prevent the global model from being biased toward the clients with dominant data volumes. The server first identifies the domains $\hat{y}_m^{D,t}$ of the participating clients by clustering the model parameters. It then aggregates the local models within the same domain to construct a model for each domain. Finally, the server builds a global model by aggregating the individual domain models. To ensure effective integration of the heterogeneous local models, the summation weights in the hierarchical aggregation process are adaptively adjusted using the proposed GMA strategy. After model aggregation, the server calculates the global class prototypes $\mathcal{P}^t$ and global domain prototypes $\mathcal{Q}^t$ based on the feature cluster information provided by the clients. Finally, the global model and global prototypes are dispatched to the clients for FL training in the next round.

Based on the separated class and domain information, the client training objective is defined as follows:

$$\begin{aligned} \theta_m^*, \mathcal{P}_m^{1,*}, \mathcal{Q}_m^{1,*} \\ = \arg \min_{\theta_m^t, \mathcal{P}_m^{1,t}, \mathcal{Q}_m^{1,t}} \sum_{\mathbf{x}_i, y_i \in \mathcal{D}_m} \mathcal{L}_m^t(F_m^t(\mathbf{x}_i; \theta_m^t, \mathcal{P}_m^t, \mathcal{Q}_m^t), y_i, \hat{y}_m^{D,t}). \end{aligned} \quad (4)$$

During server aggregation, the GHA strategy is employed to cluster the client model parameters to infer the domain of each client $\hat{y}_m^{D,t}$ and domain-specific models are then constructed using the parameters $\theta^{d,t}$ of each domain d as follows:

$$\theta^{d,t} = \sum_{\{m|\hat{y}_m^{D,t}=d\}} w_m^d \theta_m^*, \quad \text{s.t.} \quad \sum_{\{m|\hat{y}_m^{D,t}=d\}} w_m^d = 1 \quad (5)$$

where w_m^d are the summation weights calculated based on the amount of data and model gradients of the clients belonging to domain d . The global model with parameters θ^t is then obtained as follows:

$$\theta^t = \sum_d w^d \theta^{d,t}, \quad \text{s.t.} \quad \sum_d w^d = 1 \quad (6)$$

where w^d is the summation weight calculated based on the amount of data and model gradients of the corresponding domain model.

C. Client Training

The proposed client training stage and the associated network structure are shown in Fig. 2. It is assumed that the

server assigns a client $\mathcal{C}_m$ with private training dataset $\mathcal{D}_m$ to domain $\hat{y}_m^{D,t-1}$. For each training image $\mathbf{x}_i$ of class y_i , the backbone extracts a set of deep features $\mathbf{f}_i$ to represent the image. In accordance with the domain-shift problem, the data distributions of the images belonging to the same class but different domains are different because the images contain not only class information but also domain information. The extracted features $\mathbf{f}_i$ also contain both class information and domain information. Therefore, the present study uses the CGFS module to separate $\mathbf{f}_i$ into class features $\mathbf{f}_i^C$ and domain features $\mathbf{f}_i^D$. Based on these features, the CCLs and DCLs learn to predict the class and domain of the image.

The features of the images belonging to the same class should exhibit high consistency, whereas those from different classes should be clearly distinguishable. To improve the learned features, the HPCL module is employed to guide feature learning by leveraging heterogeneous dual-class prototypes and dual-domain prototypes. Accordingly, several loss functions are introduced to drive the client-side optimization. First, a feature separation loss $\mathcal{L}_S$ guides the CGFS module to disentangle the class features and domain features by enforcing orthogonality between $\mathbf{f}_i^C$ and $\mathbf{f}_i^D$. A class contrastive loss $\mathcal{L}_{\text{Con}}^C$ is then applied together with a domain contrastive loss $\mathcal{L}_{\text{Con}}^D$ to learn representative class features and domain features with the assistance of the HPCL module, class prototypes, and domain prototypes. To distinguish images of different classes, a class classification loss $\mathcal{L}_{\text{Cls}}^C$ is applied. Similarly, the model learns to identify the domain features of different domains using a domain classification loss $\mathcal{L}_{\text{Cls}}^D$. Finally, a continual regularization loss $\mathcal{L}_R$ is employed to constrain the updating of the local models by penalizing deviations of the model from the global model constructed in the previous aggregation round.

After client training, the class features and domain features at each client are clustered to represent the feature distribution of the client. Finally, the local model, clustered class features, and clustered domain features of each client are sent to the server $\mathcal{S}$. The details of the CGFS module, HPCL module, training loss functions, and feature clustering are presented in the following subsections.

1) *CGFS Module*: As formulated in (3), images belonging to the same class but different domains exhibit distinct data distributions. The domain characteristics thus affect the features representing the images and cause feature diversity within the same semantic class. To address this issue, the CGFS module is employed to separate the domain features from the class features and learn the two types of features independently. Fig. 3 shows the architecture of the CGFS module. Two feed-forward layers first project the image features $\mathbf{f}_i$ into class features $\mathbf{z}_i^C$ and domain features $\mathbf{z}_i^D$. To ensure effective feature disentanglement, the feature dimensions are explicitly separated from those associated with domain information. Specifically, if a dimension has high responses in the class features, the same dimension should have low or zero responses in the domain features and vice versa. To enforce this separation and suppress conflicting responses, the following inverse gating function $I_v(\cdot)$ is introduced:

$$I_v(\mathbf{z}) = 1 - \sigma(\mathbf{z}) \quad (7)$$

where $\sigma(\mathbf{z}) = 1/(1 + e^{-\mathbf{z}})$ is the sigmoid function. $I_v(\cdot)$ is then applied to the two feature components such that each

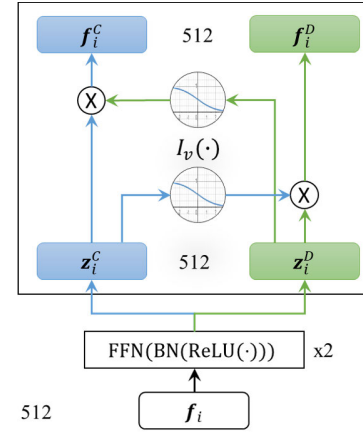


Fig. 3. Proposed CGFS module. The image features $\mathbf{f}_i$ are first projected into class features $\mathbf{z}_i^C$ and domain features $\mathbf{z}_i^D$. Then, the proposed inverse gating function $I_v(\cdot)$ enforces the feature separation by suppressing the responses of the class features based on the domain features and vice versa for the domain features. The final class features $\mathbf{f}_i^C$ and domain features $\mathbf{f}_i^D$ are generated by using $\mathbf{z}_i^C$, $\mathbf{z}_i^D$, and $I_v(\cdot)$.

component suppresses the dimensions associated with the other, formulated as follows:

$$\mathbf{f}_i^C = \mathbf{z}_i^C \cdot I_v(\mathbf{z}_i^D) \quad (8)$$

and

$$\mathbf{f}_i^D = \mathbf{z}_i^D \cdot I_v(\mathbf{z}_i^C). \quad (9)$$

To encourage the separation of the class features and domain features, the following orthogonal-based feature separation loss $\mathcal{L}_S$ is proposed:

$$\mathcal{L}_S = \sum_{\mathbf{x}_i \in \mathcal{D}_m} |\text{sim}(\mathbf{f}_i^C, \mathbf{f}_i^D)|. \quad (10)$$

In this way, the CGFS module not only enforces the separation of the class and domain features but also encourages the model to improve the learned features.

2) *HPCL Module*: Through the CGFS module, the client model learns the class features and domain features individually. To further ensure consistency among the features of images from the same class, the HPCL module leverages dual-class prototypes $\mathcal{P}_m^t = \{\mathcal{P}_m^{1,t}, \mathcal{P}_m^{2,t}\}$ and dual-domain prototypes $\mathcal{Q}_m^t = \{\mathcal{Q}_m^{1,t}, \mathcal{Q}_m^{2,t}\}$. The prototypes are provided by the server and represent the distributions of the class features and domain features, respectively. The HPCL module learns representative class features and domain features during client training, utilizing a contrastive learning approach [65], [66], [67]. In contrastive learning of the class features, the class features of each image form a positive pair set with the corresponding class prototypes and a negative pair set with the other prototypes. The contrastive loss then aligns the class features toward the prototypes of the positive pair set while simultaneously pushing them away from those in the negative pair set. The domain features are learned in a similar manner.

Let $\mathcal{P}_m^{j,t} = \{(\mathbf{p}_{m,k}^{j,t}, \mathbf{u}_{m,k}^{j,t})\}_{k=1}^{\eta_m^{j,t}}$ and $\mathcal{Q}_m^{j,t} = \{(\mathbf{q}_{m,k}^{j,t}, \mathbf{v}_{m,k}^{j,t})\}_{k=1}^{\xi_m^{j,t}}$ for $j \in \{1, 2\}$, where $\mathbf{p}_{m,k}^{j,t}$, $\mathbf{u}_{m,k}^{j,t}$, and $\eta_m^{j,t}$ are the k th class prototype, the k th prototype label, and the number of class prototypes in $\mathcal{P}_m^{j,t}$, and $\mathbf{q}_{m,k}^{j,t}$, $\mathbf{v}_{m,k}^{j,t}$, and $\xi_m^{j,t}$ are the k th domain prototype, the k th prototype label, and the number of domain prototypes in $\mathcal{Q}_m^{j,t}$, respectively. Given an image $\mathbf{x}_i$ belonging to class y_i at client $\mathcal{C}_m$, the client model backbone and CGFS module

extract the class features f_i^C and domain features f_i^D . The class features should be similar to the prototypes of the same class while being dissimilar to those of different classes. In addition, to enhance feature separation and mitigate the domain-shift problem, the class features should also be dissimilar to the domain prototypes. Thus, class feature learning is guided by the following class contrastive loss $\mathcal{L}_{\text{Con}}^C$ as follows:

$$\mathcal{L}_{\text{Con}}^C = \mathcal{L}_{\text{Con}}^{C,F} + \mathcal{L}_{\text{Con}}^{C,P} \quad (11)$$

where $\mathcal{L}_{\text{Con}}^{C,F}$ updates the class features of the client model, while $\mathcal{L}_{\text{Con}}^{C,P}$ updates the class prototypes of the first group. To compute $\mathcal{L}_{\text{Con}}^{C,F}$, a positive pair set $\Psi_i^{C,F}$ and a negative pair set $\Omega_i^{C,F}$ are first formed for f_i^C . $\Psi_i^{C,F}$ contains positive pairs from the class features f_i^C and class prototypes of the same class y_i and is defined as follows:

$$\Psi_i^{C,F} = \left\{ (f_i^C, p_{m,k}^{j,t}) \mid u_{m,k}^{j,t} = y_i \quad \forall j \in \{1, 2\} \right\}. \quad (12)$$

Meanwhile, $\Omega_i^{C,F}$ contains negative pairs that should be dissimilar. These pairs are constructed using the class feature set f_i^C , class prototypes whose labels are not y_i and the domain prototypes. $\Omega_i^{C,F}$ is defined as follows:

$$\Omega_i^{C,F} = \left\{ (f_i^C, p_{m,k}^{j,t}) \mid u_{m,k}^{j,t} \neq y_i \quad \forall j \in \{1, 2\} \right\} \cup \left\{ (f_i^C, q_{m,r}^{j,t}) \mid \forall j \in \{1, 2\} \right\}. \quad (13)$$

Let $\text{sim}(v_1, v_2)$ be the cosine similarity between two vectors v_1 and v_2 and let $S(v_1, v_2) = \exp((\text{sim}(v_1, v_2))/\tau)$, where $\tau = 0.5$ is the temperature parameter. The class contrastive loss $\mathcal{L}_{\text{Con}}^{C,F}$ is then computed based on $\Psi_i^{C,F}$ and $\Omega_i^{C,F}$ as follows:

$$\mathcal{L}_{\text{Con}}^{C,F} = - \sum_{x_i \in \mathcal{D}_m} \log \frac{\sum_{(\mu_1, \mu_2) \in \Psi_i^{C,F}} S(\mu_1, \mu_2)}{\sum_{(v_1, v_2) \in \Psi_i^{C,F} \cup \Omega_i^{C,F}} S(v_1, v_2)}. \quad (14)$$

To compute $\mathcal{L}_{\text{Con}}^{C,P}$, a positive pair set $\Psi_i^{C,P}$ and negative pair set $\Omega_i^{C,P}$ are first formed. The positive pair set $\Psi_i^{C,P}$ consists of positive pairs formed between the first-group class prototype $p_{m,k}^{1,t}$ and both the class features f_i^C and the remaining class prototypes of the first and second groups that share the same class label y_i . It is defined as follows:

$$\Psi_i^{C,P} = \left\{ (p_{m,k}^{1,t}, f_i^C) \mid u_{m,k}^{1,t} = y_i \right\} \cup \left\{ (p_{m,k}^{1,t}, p_{m,r}^{1,t}) \mid u_{m,k}^{1,t} = y_i, u_{m,r}^{1,t} = y_i, k \neq r \right\} \cup \left\{ (p_{m,k}^{1,t}, p_{m,s}^{2,t}) \mid u_{m,k}^{1,t} = y_i, u_{m,s}^{2,t} = y_i \right\}. \quad (15)$$

The negative pair set $\Omega_i^{C,P}$ consists of negative pairs formed between the first-group class prototype $p_{m,k}^{1,t}$ and the remaining class prototypes of the first or the second group whose labels are not y_i , together with the domain prototypes. It is defined as follows:

$$\Omega_i^{C,P} = \left\{ (p_{m,k}^{1,t}, p_{m,r}^{j,t}) \mid u_{m,k}^{1,t} = y_i, u_{m,r}^{j,t} \neq y_i \quad \forall j \in \{1, 2\} \right\} \cup \left\{ (p_{m,k}^{1,t}, q_{m,s}^{j,t}) \mid \forall j \in \{1, 2\} \right\}. \quad (16)$$

Then, $\mathcal{L}_{\text{Con}}^{C,P}$ is computed based on $\Psi_i^{C,P}$ and $\Omega_i^{C,P}$ as follows:

$$\mathcal{L}_{\text{Con}}^{C,P} = - \sum_{x_i \in \mathcal{D}_m} \log \frac{\sum_{(\mu_1, \mu_2) \in \Psi_i^{C,P}} S(\mu_1, \mu_2)}{\sum_{(v_1, v_2) \in \Psi_i^{C,P} \cup \Omega_i^{C,P}} S(v_1, v_2)}. \quad (17)$$

The class contrastive loss guides the client model to reduce the distance between class features belonging to the same

class while increasing the distance between class features of different classes and all the domain-related features. Thus, the client model learns class features that are both representative of their semantic categories and highly discriminative from other classes, leading to an improved classification performance.

Similarly, to learn the domain features, the domain contrastive loss $\mathcal{L}_{\text{Con}}^D$ is calculated as follows:

$$\mathcal{L}_{\text{Con}}^D = \mathcal{L}_{\text{Con}}^{D,F} + \mathcal{L}_{\text{Con}}^{D,P} \quad (18)$$

where $\mathcal{L}_{\text{Con}}^{D,F}$ updates the domain features of the client model and $\mathcal{L}_{\text{Con}}^{D,P}$ updates the domain prototypes of the first group. To compute $\mathcal{L}_{\text{Con}}^{D,F}$, a positive pair set $\Psi_i^{D,F}$ and negative pair set $\Omega_i^{D,F}$ for f_i^D are formed. $\Psi_i^{D,F}$ consists of positive pairs of the domain features and domain prototypes of domain $\hat{y}_m^{D,t-1}$ as follows:

$$\Psi_i^{D,F} = \left\{ (f_i^D, q_{m,k}^{j,t}) \mid v_{m,k}^{j,t} = \hat{y}_m^{D,t-1} \quad \forall j \in \{1, 2\} \right\}. \quad (19)$$

$\Omega_i^{D,F}$ consists of negative pairs formed by the domain features f_i^D , domain prototypes with labels different to $\hat{y}_m^{D,t-1}$, and class prototypes as follows:

$$\Omega_i^{D,F} = \left\{ (f_i^D, q_{m,k}^{j,t}) \mid v_{m,k}^{j,t} \neq \hat{y}_m^{D,t-1} \quad \forall j \in \{1, 2\} \right\} \cup \left\{ (f_i^D, p_{m,r}^{j,t}) \mid \forall j \in \{1, 2\} \right\} \quad (20)$$

based on $\Psi_i^{D,F}$ and $\Omega_i^{D,F}$, the domain contrastive loss $\mathcal{L}_{\text{Con}}^{D,F}$ is computed as follows:

$$\mathcal{L}_{\text{Con}}^{D,F} = - \sum_{x_i \in \mathcal{D}_m} \log \frac{\sum_{(\mu_1, \mu_2) \in \Psi_i^{D,F}} S(\mu_1, \mu_2)}{\sum_{(v_1, v_2) \in \Psi_i^{D,F} \cup \Omega_i^{D,F}} S(v_1, v_2)}. \quad (21)$$

To compute $\mathcal{L}_{\text{Con}}^{D,P}$, a positive pair set $\Psi_i^{D,P}$ and a negative pair set $\Omega_i^{D,P}$ are first formed. $\Psi_i^{D,P}$ contains positive pairs of the domain prototypes $q_{m,k}^{1,t}$ of the first group with respect to the domain features f_i^D and the remaining domain prototypes of the first and second groups having the same domain label $\hat{y}_m^{D,t-1}$. It is defined as follows:

$$\Psi_i^{D,P} = \left\{ (q_{m,k}^{1,t}, f_i^D) \mid v_{m,k}^{1,t} = \hat{y}_m^{D,t-1} \right\} \cup \left\{ (q_{m,k}^{1,t}, q_{m,r}^{1,t}) \mid v_{m,k}^{1,t} = \hat{y}_m^{D,t-1}, v_{m,r}^{1,t} = \hat{y}_m^{D,t-1}, k \neq r \right\} \cup \left\{ (q_{m,k}^{1,t}, q_{m,s}^{2,t}) \mid u_{m,k}^{1,t} = \hat{y}_m^{D,t-1}, v_{m,s}^{2,t} = \hat{y}_m^{D,t-1} \right\}. \quad (22)$$

Meanwhile, $\Omega_i^{D,P}$ consists of the negative pairs of the domain prototypes $q_{m,k}^{1,t}$ of the first group with respect to the remaining domain prototypes whose labels are not $\hat{y}_m^{D,t-1}$, together with the class prototypes. It is defined as follows:

$$\Omega_i^{D,P} = \left\{ (q_{m,k}^{1,t}, q_{m,r}^{j,t}) \mid v_{m,k}^{1,t} = \hat{y}_m^{D,t-1}, v_{m,r}^{j,t} \neq \hat{y}_m^{D,t-1} \quad \forall j \in \{1, 2\} \right\} \cup \left\{ (q_{m,k}^{1,t}, p_{m,s}^{j,t}) \mid \forall j \in \{1, 2\} \right\}. \quad (23)$$

$\mathcal{L}_{\text{Con}}^{D,P}$ is then computed based on $\Psi_i^{D,P}$ and $\Omega_i^{D,P}$ as follows:

$$\mathcal{L}_{\text{Con}}^{D,P} = - \sum_{x_i \in \mathcal{D}_m} \log \frac{\sum_{(\mu_1, \mu_2) \in \Psi_i^{D,P}} S(\mu_1, \mu_2)}{\sum_{(v_1, v_2) \in \Psi_i^{D,P} \cup \Omega_i^{D,P}} S(v_1, v_2)}. \quad (24)$$

With dual prototypes and contrastive losses, the HPCL module enforces consistent, representative, and discriminative class features for images of the same class across different domains, thereby mitigating the domain-shift problem.

3) *Client Training Loss*: This section introduces the remaining losses used to drive client model training. First, to identify the class of an image x_i , the CCLs make a prediction $\hat{y}_i^C$ using the class features f_i^C . The class classification loss $\mathcal{L}_{\text{Cls}}^C$ is then computed as follows:

$$\mathcal{L}_{\text{Cls}}^C = - \sum_{x_i \in \mathcal{D}_m} y_i \log \hat{y}_i^C. \quad (25)$$

Similarly, the DCLs predict the domain label $\hat{y}_i^D$ using the domain features f_i^D , and the domain classification loss $\mathcal{L}_{\text{Cls}}^D$ is computed as follows:

$$\mathcal{L}_{\text{Cls}}^D = - \sum_{x_i \in \mathcal{D}_m} \hat{y}_m^{D,t-1} \log \hat{y}_i^D. \quad (26)$$

To regularize the update gradients of the client models, the continual regularization loss $\mathcal{L}_R$ is calculated based on [14] as follows:

$$\mathcal{L}_R = \lambda \|(\theta_m^t - \theta^{t-1})\|_2^2 \quad (27)$$

where θ^{t-1} represents the parameters of the global model built in round $t-1$.

Finally, the client training loss $\mathcal{L}_m^t$ is computed as follows:

$$\mathcal{L}_m^t = \mathcal{L}_{\text{Cls}}^C + \mathcal{L}_{\text{Cls}}^D + \mathcal{L}_{\text{Con}}^C + \mathcal{L}_{\text{Con}}^D + \mathcal{L}_S + \mathcal{L}_R. \quad (28)$$

4) *Client Feature Clustering*: After client training, the class and domain features are clustered to represent the feature distributions of the clients. Clustering is performed based on the neighbor relationship among the features. In particular, for each class c , clustering is performed on the class features of the images belonging to the same class. The centers of the resulting feature clusters are then obtained by averaging the features within each cluster to form a set $\mathcal{H}_m^{C,c} = \{\mathbf{h}_{m,j}^{C,c}\}_{j=1}^{\gamma_c}$ in client C_m , where γ_c is the number of clusters of class c . Finally, a set $\mathcal{H}_m^C$ is formed as follows:

$$\mathcal{H}_m^C = \{\mathcal{H}_m^{C,1}, \dots, \mathcal{H}_m^{C,c}, \dots, \mathcal{H}_m^{C,N^C}\} \quad (29)$$

where N^C is the number of classes at the client.

Similarly, clustering is performed on the domain features of client C_m to obtain a set of cluster centers as follows:

$$\mathcal{H}_m^D = \{\mathbf{h}_{m,j}^D\}_{j=1}^{\kappa_m} \quad (30)$$

where κ_m is the number of domain feature clusters at client C_m . Since the images of a client belong to the same domain $\hat{y}_m^{D,t-1}$, the level of $\mathcal{H}_m^{D,d}$ is omitted to simplify the equation.

Finally, the local model, class feature clusters $\mathcal{H}_m^C$, and domain feature clusters $\mathcal{H}_m^D$ of each client are uploaded to the server for aggregation.

D. Server Aggregation

To prevent the global model from being biased toward clients with dominant data volumes, the aggregation process is performed with the assistance of the GHA strategy. Different data distributions arising from domain shift lead to different optimal parameter spaces for different domains. To address this issue, the server clusters the client model parameters based on their cosine similarity [68], and clients assigned to the same cluster are considered to belong to the same domain since their local models exhibit similar parameter characteristics. Based on the clustering results, the domain $\hat{y}_m^{D,t}$ of each client is

estimated and assigned for client training in round $t+1$. A domain model $\theta^{d,t}$ is built for each domain d by aggregating the local models belonging to the same domain using (5). The server then builds the global model θ^t by aggregating all the domain models using (6).

To effectively integrate the different local models, the summation weights w_m^d and w^d of the hierarchical aggregation process are adjusted in accordance with the GMA strategy, as described next. After model aggregation, the server calculates the global class prototypes and global domain prototypes using the feature clusters $\mathcal{H}_m^C$ and $\mathcal{H}_m^D$ received from the clients. Finally, the global model θ^t , global prototypes $\mathcal{P}^t$ and $\mathcal{Q}^t$, and domain labels $\hat{y}_m^{D,t}$ are dispatched to the clients for FL training in the next round.

1) *GMA*: Due to the domain-shift problem, the parameters of the local models are diverse. When aggregating the local models, the summation weights of the models play important roles in determining the quality of the global model. However, if the aggregation weights are established solely on the client data volume, the global model may become biased toward the clients with dominant data distributions. To mitigate this effect and more effectively aggregate the different local models, the difference between the local model and the global model in round $t-1$ is incorporated into the weighting strategy. This difference reflects the gradient update direction of each client and is used to adaptively determine the aggregation weights.

The gradient information can be characterized by both its angle and magnitude. If a local model has large gradients in both angle and magnitude with respect to the global model constructed in the previous round, this indicates that the global model does not adequately fit the private data, and thus, substantial parameter updates of the local model occur during client training. Such a model, therefore, contains additional information not captured in the previous global model and should thus receive a higher aggregation weight. Consequently, in addition to the amount of data possessed by the clients, the summation weights used in the aggregation process also take into account both the angular deviation and magnitude difference between each local model and the previous global model.

Let $\phi_m^t = 1 - \text{sim}(\theta_m^{t*}, \theta^{t-1})$ and $\Delta_m^t = |\theta_m^{t*} - \theta^{t-1}|$ represent the angle and magnitude of the local model of client C_m in round t with respect to the global model in round $t-1$, respectively. For each local model in domain d , the summation weight w_m^d used to build the domain models in (5) is based on the angular deviation ϕ_m^t , magnitude difference Δ_m^t , and amount of data at the client as follows:

$$w_m^d = \frac{N_m}{\sum_{\{m|\hat{y}_m^{D,t}=d\}} N_m} + \alpha \frac{\phi_m^t \cdot \Delta_m^t}{\sum_{\{m|\hat{y}_m^{D,t}=d\}} \phi_m^t \cdot \Delta_m^t} \quad (31)$$

where α is a hyperparameter used to adjust the influence of the gradient information. All w_m^d in domain d is then normalized to a summation of 1. Afterward, the domain model of each domain d is built using (5) based on w_m^d .

Similarly, to aggregate the domain models and build the global model in (6), the summation weight w^d for each domain d is computed as follows. Let $N^{d,t} = \sum_{\{m|\hat{y}_m^{D,t}=d\}} N_m$ be the amount of data in domain d , and $\phi^{d,t} = 1 - \text{sim}(\theta^{d,t}, \theta^{t-1})$ and $\Delta^{d,t} = |\theta^{d,t} - \theta^{t-1}|$ represent the angle and magnitude of the difference between the domain model of domain d and the

global model built in round $t-1$, respectively. The summation weight w^d for each domain model is computed as follows:

$$w^d = \frac{N^{d,t}}{\sum_d N^{d,t}} + \alpha \frac{\phi^{d,t} \cdot \Delta^{d,t}}{\sum_d \phi^{d,t} \cdot \Delta^{d,t}}. \quad (32)$$

All w^d values are then normalized to a summation of 1, and the global model is constructed using (6).

2) *Global Prototypes Calculation*: After model aggregation, the server derives the global class and domain prototypes using the feature clusters collected from each client, thereby capturing the joint feature distributions of all the domains. As remarked in [3], the class prototypes should not only preserve the diversity of the class features but also serve as consistent anchors to guide feature learning for all the clients. Therefore, the class prototypes of the first group are built by clustering to preserve the variety of the class features, while those of the second group are built based on the average of the feature clusters $\mathcal{H}_m^c$ from all the clients to provide a consistent semantic anchor for each class. For each class c , the class feature clusters $\{\mathcal{H}_m^{c,d} | \hat{y}^{D,t} = d\}$ of the clients in domain d are grouped by clustering to form a subset $\mathcal{G}^{c,d}$ of the class prototypes, where the cluster centers $\mathbf{g}_i^{c,d}$ serve as prototype representatives as follows:

$$\mathcal{G}^{c,d} = \left\{ \left(\mathbf{g}_i^{c,d}, c \right) \right\}_{i=1}^{\Gamma^{c,d}} \quad (33)$$

where $\Gamma^{c,d}$ is the number of prototypes for class c in domain d . The class prototypes $\mathcal{P}^{1,t}$ of the first group are then built by taking the union of $\mathcal{G}^{c,d}$ for all classes and domains, as shown in the following:

$$\mathcal{P}^{1,t} = \bigcup_{c,d} \mathcal{G}^{c,d}. \quad (34)$$

The class prototypes $\mathcal{P}^{2,t}$ of the second group are built by averaging $\mathbf{g}_i^{c,d}$ of the same class c as follows:

$$\mathcal{P}^{2,t} = \bigcup_c \{(\bar{\mathbf{g}}^c, c)\} \quad (35)$$

where $\bar{\mathbf{g}}^c$ is the average of $\mathbf{g}_i^{c,d}$ and is defined as follows:

$$\bar{\mathbf{g}}^c = \frac{1}{\sum_d |\mathcal{G}^{c,d}|} \sum_d \sum_{\mathbf{g}_i^{c,d} \in \mathcal{G}^{c,d}} \mathbf{g}_i^{c,d}. \quad (36)$$

Similarly, for each domain d , the domain feature clusters $\{\mathcal{H}_m^D | \hat{y}^{D,t} = d\}$ are grouped via clustering, and the resulting cluster centers $\mathbf{g}_i^d$ form the subset $\mathcal{G}^d$ of domain prototypes as follows:

$$\mathcal{G}^d = \left\{ \left(\mathbf{g}_i^d, d \right) \right\}_{i=1}^{\Gamma^d} \quad (37)$$

where Γ^d is the number of prototypes in domain d . The domain prototypes $\mathcal{Q}^{1,t}$ and $\mathcal{Q}^{2,t}$ are then built using $\mathcal{G}^d$ as follows:

$$\mathcal{Q}^{j,t} = \bigcup_d \mathcal{G}^d \quad \forall j \in \{1, 2\}. \quad (38)$$

The global class prototypes and global domain prototypes are then dispatched to the clients along with the global model for client training in round $t+1$.

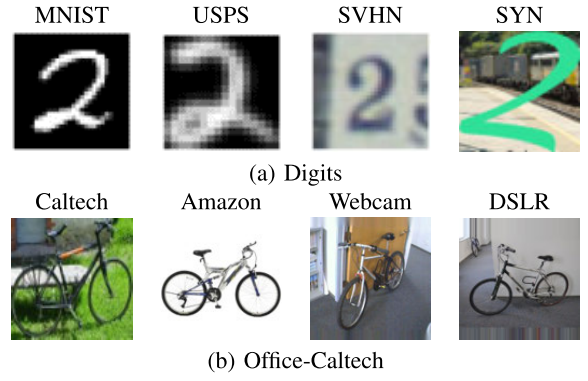


Fig. 4. Sample images in (a) Digits dataset and (b) Office-Caltech dataset. Each dataset contains four domains with different styles of appearances, which leads to the domain-shift problem.

IV. EXPERIMENTS

A. Dataset and Settings

Following recent state-of-the-art work [3], [4], the performance of the proposed method was evaluated on two FL datasets, Digits and Office-Caltech, under both a controlled domain-shift setting in FedHeal [4] and a skewed domain-shift setting in FPL [3].

As shown in Fig. 4(a), the Digits dataset comprised images of digits 0–9 acquired from four different domains: MNIST [69], USPS [70], SVHN [71], and SYN [72]. Meanwhile, the Office-Caltech dataset consisted of ten overlapping classes of office items from Caltech256 [73] and Office31 [74]. As illustrated in Fig. 4(b), the images within the dataset were distributed across four domains: Caltech, Amazon, Webcam, and DSLR.

To simulate the domain-shift problem, each client was randomly assigned to a domain, and each client randomly sampled the images from the assigned domain.

In the controlled domain-shift setting [4], each domain was assigned to the same number of clients. The controlled setting aimed to evaluate the ability of the proposed FL framework to mitigate the domain-shift problem under controlled conditions.

In contrast, in the skewed domain-shift setting [3], each domain was randomly assigned to an unequal number of clients. That is, some domains were assigned to more clients, while others were assigned to fewer clients. In this case, the domains with larger client populations tended to dominate the model aggregation process, further intensifying the domain-shift problem. As a result, the skewed setting better reflected real-world FL deployments, in which the data distributions are naturally imbalanced. The details of the two settings are described as follows.

In the controlled domain-shift setting [4], both datasets were trained for 200 communication rounds. A total of 20 clients were used, evenly distributed across the four domains, and each client performed 10 local training epochs per round. For the Digits dataset, each client randomly sampled 1% of the data from the target domain, and the batch size was set to 64. In contrast, the Office-Caltech clients randomly sampled 10% of the data from the target domain, and the batch size was set to 16. In the skewed domain-shift setting [3], 20 clients were randomly assigned to different domains for the Digits dataset, and each client randomly selected 1% of the data from the target domain. For the Office-Caltech dataset, ten clients

TABLE I
ABLATION STUDY RESULTS ON DIGITS DATASET AND OFFICE-CALTECH DATASET

$\mathcal{L}_{cont}$	CGFS	HPCL	GHA	Digits					Office-Caltech				
				MNIST	USPS	SVHN	SYN	Average	Caltech	Amazon	Webcam	DSLR	Average
V				97.80	<u>92.87</u>	73.25	40.05	75.99	61.16	75.26	44.83	30.00	52.81
V	V			97.66	92.13	73.07	41.49	76.09	61.16	75.26	46.55	30.00	53.24
V	V	V		98.10	92.92	74.61	41.39	76.76	63.39	<u>75.79</u>	<u>60.34</u>	<u>36.67</u>	59.05
V	V			98.32	92.48	<u>79.89</u>	<u>53.48</u>	<u>81.04</u>	65.18	77.89	53.45	53.33	<u>62.46</u>
V	V	V	V	98.17	92.23	83.16	63.43	84.25	<u>63.84</u>	71.05	65.52	53.33	63.44

were randomly assigned to different domains, and each client randomly selected 20% of the data from the target domain. For both datasets, training proceeded for 100 communication rounds, and each client performed 10 local training epochs per round with a batch size of 64.

The implementation details are presented in the Supplementary Material.

B. Ablation Study

The experiments commenced by investigating the effectiveness of each module in the proposed FL framework. Evaluations were performed on both datasets (Digits and Office-Caltech) under the skewed domain-shift setting [3]. Table I shows the ablation study results for each module. As shown in the first row, the baseline model retrained using FedAvg [1] was severely affected by domain shift and achieved a poor performance on both the SYN images of the Digits dataset and the Webcam and DSLR images of the Office-Caltech dataset. Fig. 4 shows that the appearance of the images in these domains differs substantially from those in the other domains, which leads to a more pronounced domain-shift effect and consequently a degraded performance. Referring to the second row in Table I, the continual loss helped constrain the local model gradients and thus slightly improved the overall performance. However, the domain-shift problem persisted. By separating the class and domain features using the proposed CGFS module, the performance of the proposed FL framework for the Webcam and DSLR images on the Office-Caltech dataset significantly improved, as shown in the third row of the table. However, possibly due to the substantial appearance gap between the SYN images and the other domains on the Digits dataset, the CGFS module provided only limited benefit in the SYN domain. When the HPCL module was further introduced, it not only guided the model to learn more consistent features of the same class but also improved the feature discriminability between different classes, as shown in the fourth row of Table I. Therefore, the proposed FL framework significantly improved the FL performance on the SVHN and SYN domains of the Digits dataset, as well as on the DSLR domain of the Office-Caltech dataset. Finally, the proposed GHA strategy helped the server conduct more effective aggregation of the local models from different domains, leading to further performance improvements on the challenging SYN domain of the Digits dataset and Webcam domain of the Office-Caltech dataset, as shown in the last row of Table I.

It is worth noting that GHA aims to prevent the global model from being biased toward certain domains. Therefore, after applying GHA, some domains that previously exhibited a weaker performance showed improvement, while others that were originally favored by aggregation experienced slight

performance reductions due to a more balanced contribution of the different domains. Similar observations were also reported in [4]. Overall, each of the proposed modules helped alleviate the domain-shift problem in FL, resulting in improved performance in previously underperforming domains and stronger global accuracy.

Further comparisons of alternative design configurations are available in the Supplementary Material.

C. Comparison With State-of-the-Art Methods

This section compares the performance of the proposed FL method with various state-of-the-art methods under both a controlled domain-shift setting [4] and a skewed domain-shift setting [3]. For the methods marked with an asterisk (*) in Tables II and III, the reported performance was obtained from FL models trained using their official implementations. The performance of the other methods is reported in [3] and [4].

Table II shows the performance of the various methods when applied to the Digits dataset and Office-Caltech dataset under the controlled domain-shift setting [4]. The best-performing method is highlighted in bold, and the second-best method is underlined. The table is divided into three sections. The performance of the nonprototype-based methods is shown in the first section, while that of the prototype-based methods [3], [11], [12], [13], [56] is presented in the second section. The performance of the proposed method is reported in the third section.

Due to the domain-shift problem, most of the methods performed poorly on domains whose images differed substantially from those of the other domains in the dataset, such as SYN in Digits and Webcam in Office-Caltech. By introducing prototypes to represent the features across clients, FedProto [11] achieved a slight performance improvement compared to the FedAvg baseline. However, the substantial visual differences across domains resulted in heterogeneous feature representations, and the class prototypes obtained by simple feature averaging therefore failed to accurately capture the true class distributions. In addition, simply aligning the features to the prototypes was ineffective in separating the different classes.

To enhance the separation among prototypes of different classes, FedTGP [13] performed contrastive learning on the server. However, it struggled to learn universal representations for different domains and thus performed poorly on domains such as SYN and Caltech. By using cluster prototypes, FPL [3] better captured the domain-specific feature distributions. This then enabled the local models to more effectively learn the object characteristics through contrastive learning with prototypes. Therefore, FPL [3] greatly improved the performance compared to other methods and achieved the second-best performance in both datasets. In the proposed framework, the

TABLE II

COMPARISON OF COMPETING METHODS ON DIGITS DATASET AND OFFICE-CALTECH DATASET UNDER CONTROLLED DOMAIN-SHIFT SETTING [4]

Method	Digit					Office-Caltech				
	MNIST	USPS	SVHN	SYN	Average	Caltech	Amazon	Webcam	DSLR	Average
FedAvg [1]	89.84	93.25	79.54	41.35	76.00	58.57	72.63	45.52	56.67	58.35
FedProx [14]	90.27	93.93	80.04	42.82	76.76	60.27	69.90	45.52	58.00	58.42
Scaffold [15]	94.15	94.44	76.87	44.22	77.42	59.55	69.37	46.21	59.33	58.62
MOON [16]	90.46	92.65	80.48	40.58	76.04	60.63	74.00	46.90	59.33	60.21
FedDyn [18]	91.23	92.36	80.15	41.55	76.32	59.20	71.16	48.62	62.00	60.24
FedAvg + AFL [22]	90.59	95.83	75.13	44.42	76.49	57.50	64.21	48.28	65.37	58.83
FedAvg + q-FFL [23]	91.44	94.10	76.33	44.48	76.59	53.39	60.00	51.72	64.01	57.28
Ditto [40]	90.59	92.98	79.20	41.89	76.16	56.25	58.00	63.45	<u>70.00</u>	61.92
FedFV [7]	91.76	94.70	77.26	44.14	76.97	55.36	62.95	60.00	71.33	62.41
FedAvg + FedHEAL [4]	90.27	<u>95.69</u>	79.94	46.45	78.09	59.28	67.90	66.21	66.00	64.85
FedAvg + FedRDN* [36]	97.29	93.97	80.63	42.64	78.63	<u>64.29</u>	76.32	68.97	60.00	67.40
FedAWA* [27]	95.98	94.27	79.46	47.81	79.38	58.48	67.37	<u>70.69</u>	53.33	62.47
FedProto [11]	89.99	92.90	81.09	40.93	76.23	62.23	71.48	60.34	42.67	59.18
FedTGP* [13]	96.94	76.91	69.95	12.34	64.04	28.66	44.95	51.03	59.33	46.00
FPL* [3]	97.98	93.72	<u>86.12</u>	<u>46.92</u>	<u>81.19</u>	64.73	<u>78.42</u>	65.52	60.00	<u>67.17</u>
FedProc [56]	91.86	91.16	78.54	39.87	75.36	55.98	60.21	46.90	46.00	52.27
FedPLVM* [12]	96.21	86.40	85.81	37.06	76.37	54.46	70.53	58.62	53.33	59.24
Ours	<u>97.44</u>	94.17	86.75	49.80	82.04	64.73	83.16	74.14	60.00	70.51

TABLE III

COMPARISONS OF COMPETING METHODS ON DIGITS DATASET AND OFFICE-CALTECH DATASET UNDER SKEWED DOMAIN-SHIFT SETTING [3]

Method	Digit					Office-Caltech				
	MNIST	USPS	SVHN	SYN	Average	Caltech	Amazon	Webcam	DSLR	Average
FedAvg [1]	98.14	90.85	76.56	55.01	80.14	60.15	75.44	45.86	36.00	54.36
FedProx [14]	98.11	90.24	77.01	56.66	80.50	60.21	77.44	48.62	37.33	55.90
MOON [16]	97.44	92.15	77.62	38.79	76.50	56.19	71.54	41.04	30.22	49.74
FedDyn [18]	98.01	91.00	78.95	54.22	80.54	61.64	75.54	48.28	35.56	55.25
FedOPT [17]	96.23	91.80	73.03	57.85	79.72	56.31	56.74	63.33	48.89	56.31
FedAvg + FedRDN* [36]	97.25	91.63	71.79	40.55	75.31	58.48	63.16	41.38	<u>53.33</u>	54.09
FedAWA* [27]	97.08	93.37	74.37	50.15	78.74	57.14	53.68	63.79	40.00	53.65
FedProto [11]	98.30	<u>92.44</u>	80.35	53.58	81.16	<u>64.02</u>	79.37	50.17	40.33	58.47
FedTGP* [13]	95.96	75.81	65.41	12.13	62.33	42.56	68.95	<u>63.79</u>	67.5	60.70
FPL [3]	98.31	92.71	80.27	<u>61.20</u>	<u>83.12</u>	63.39	<u>79.26</u>	55.86	48.00	<u>61.63</u>
FedProc [56]	97.86	88.99	78.90	45.84	77.89	58.07	73.65	42.76	30.22	51.17
FedPLVM* [12]	96.81	89.29	<u>82.17</u>	40.25	77.13	64.73	78.42	56.90	43.33	60.85
Ours	98.17	92.23	83.16	63.43	84.25	63.84	71.05	65.52	<u>53.33</u>	63.44

domain features were disentangled from the class features by the CGFS module, and the model was guided to learn consistent features of the same class by the HPCL module. In addition, the GHA strategy improved the ability of the server to effectively aggregate information from different domains. As a result, the proposed method achieved the best overall performance on both datasets.

Table III compares the performance of the various methods in the two datasets under the skewed domain-shift setting [3]. Consistent with the results in Table II, most of the methods performed poorly on the SYN, Webcam, and DSLR images due to the domain-shift problem. However, through the guidance of feature prototypes, the prototype-based methods [3], [11], [12] greatly improved the performance in the SYN, Webcam, and DSLR images while maintaining the model performance in the other domains. Thus, these methods showed better performance compared to the other methods, with FPL [3] achieving the second-best overall performance.

Beyond leveraging feature prototypes in the HPCL module, the proposed framework explicitly disentangled the class and domain features through the CGFS module and incorporated effective hierarchical aggregation via GHA to reduce the aggregation bias. As a result, it achieved the best overall performance on both datasets under both settings. Regarding the complexity, the proposed method has a similar complexity to that of the other prototype-based methods, albeit with a slightly higher server complexity due to the hierarchical model

aggregation. More detailed analysis is presented in the Supplementary Material. These comprehensive comparisons against state-of-the-art methods clearly demonstrate the superiority of the proposed FL framework under heterogeneous domain conditions.

D. Visualization

Fig. 5 presents t-SNE [75] visualizations of the feature distributions learned by FedAvg [1], FPL [3], FedTGP [13], and the proposed method on the Digits and Office-Caltech datasets across different domains. Each point represents the class feature of a testing image from the target dataset, and the color of the point represents the domain of the image. As shown in Fig. 5(a), the FedAvg model separated the features from different domains into distinct regions. This indicates that the domain identity can be inferred directly from the class features. However, it also implies that the class features are affected by domain shift and are determined mainly by domain information rather than class information. Similarly, because prototype-based methods align features solely based on class information, a noticeable separation persists among points of different colors in Fig. 5(b) and (c) for FPL [3] and FedTGP [13], respectively. In contrast, the proposed method separates domain-related information from class-discriminative information through the CGFS module and further guides feature alignment using the HPCL module. Meanwhile, the GHA

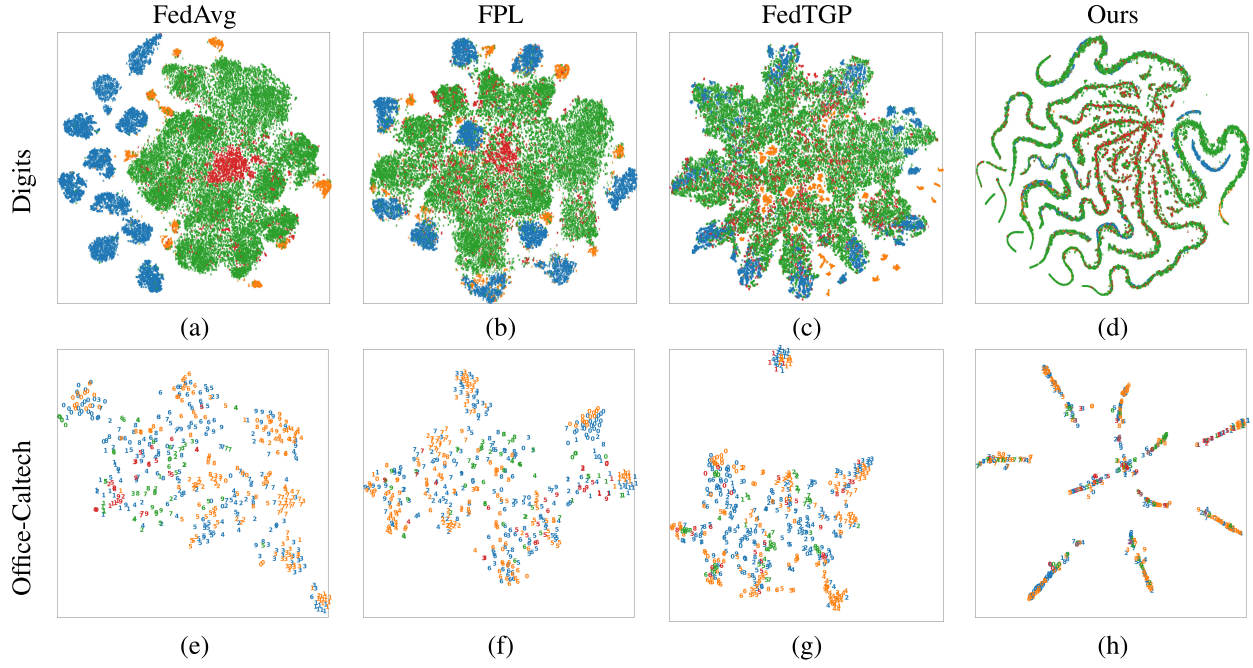


Fig. 5. t-SNE visualization results with different colors representing different domains. (a) FedAvg on Digits dataset. (b) FPL on Digits dataset. (c) FedTGP on Digits dataset. (d) Proposed method on Digits dataset. (e) FedAvg on Office-Caltech dataset. (f) FPL on Office-Caltech dataset. (g) FedTGP on Office-Caltech dataset. (h) Proposed method on Office-Caltech dataset.

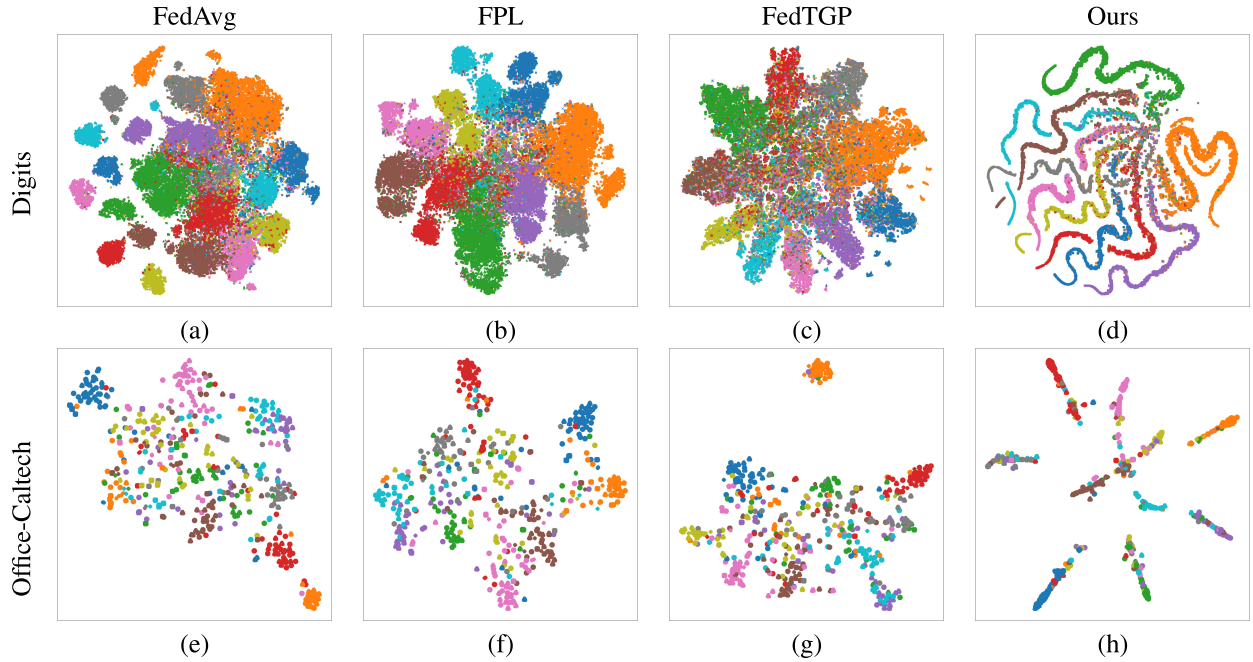


Fig. 6. t-SNE visualization results with different colors representing different classes. (a) FedAvg on Digits dataset. (b) FPL on Digits dataset. (c) FedTGP on Digits dataset. (d) Proposed method on Digits dataset. (e) FedAvg on Office-Caltech dataset. (f) FPL on Office-Caltech dataset. (g) FedTGP on Office-Caltech dataset. (h) Proposed method on Office-Caltech dataset.

strategy enables the server to effectively aggregate heterogeneous local models across domains. As a result, class features from different domains become well mixed, making it difficult to infer domain identity from their distributions. This indicates that the learned class features contain minimal domain-specific information, demonstrating the successful disentanglement of the class and domain features, as illustrated in Fig. 5(d). Similar tendencies can be observed in Fig. 5(e)–(h), where the

proposed method again achieves superior cross-domain feature alignment compared with the competing methods.

Fig. 6 shows the t-SNE [75] feature visualization results learned by four methods across different classes. In each figure, the points represent the class features of a testing image selected from the target dataset, and the point colors indicate the class label of the image. As shown in Fig. 6(a), for the FedAvg model, points of the same color are separated

into distinct regions. This indicates that the class features of samples from the same class but different domains are inconsistent, reflecting strong domain dependency. Consequently, the learned representations encode domain-specific characteristics rather than purely class-discriminative information. By leveraging class prototypes to guide feature learning, prototype-based methods such as FPL [3] and FedTGP [13] achieve a better alignment of points with the same class, as shown in Fig. 6(b) and (c). However, the learned representations still retain noticeable domain-specific characteristics. That is, some points belonging to the same class remain split across multiple regions, while points from different classes occasionally overlap, indicating an incomplete separation of the class information from the domain information. By contrast, the proposed method explicitly disentangles the class and domain representations through the CGFS and HPCL modules, while the GHA strategy enables the server to effectively integrate information across domains. As a result, the learned class features for images belonging to the same class across different domains become highly consistent, while those of different classes remain clearly separate, as shown in Fig. 6(d). Similar observations can be made in Fig. 6(e)–(h) for the three compared methods and the proposed method in the Office-Caltech dataset. Overall, the t-SNE visualizations demonstrate that the proposed method learns more compact and discriminative class features than FedAvg and prototype-based methods, ultimately contributing to substantial performance improvements in both datasets.

V. CONCLUSION

This article presents a class- and domain-aware framework designed to mitigate the domain-shift problem in FL settings. During client training, two newly proposed modules (CGFS and HPCL) are used to explicitly disentangle and learn the class and domain features separately, thereby reducing feature inconsistency across the clients. At the server side, a GHA strategy is employed to facilitate an effective aggregation of the local models by accounting for the client data volumes and domain differences. Extensive experiments were performed on two benchmark datasets. The average accuracy of the proposed method was 0.8% higher than the second-best method on the Digits dataset and 3% higher than the second-best method on the Office-Caltech dataset under the controlled setting. Moreover, in the more challenging skewed setting, the proposed method achieved 1.1% and 1.8% improvements compared to the second-best method on the Digits dataset and Office-Caltech dataset, respectively. Such results show that the proposed method consistently outperforms state-of-the-art approaches under both controlled and skewed domain-shift settings.

There are two promising directions for future work. First, to further reduce the computational overhead of the hierarchical model aggregation process, alternative and more efficient representations of model discrepancy, beyond the cosine similarity of the local model parameters adopted in the present study, will be investigated. Second, because FL aggregates models between clients without sharing data, it is well-suited to train AI models between hospitals due to its inherent privacy-preserving characteristics. However, domain shift commonly occurs in real clinical environments because

medical data (e.g., CT or pathology images) are acquired using heterogeneous imaging protocols and devices between hospitals. Therefore, the proposed framework will be specifically adapted to clinical scenarios to enable the development of more robust and reliable federated models for CT images or federated models for pathology images in clinical deployment.

ACKNOWLEDGMENT

The authors would like to thank the National Center for High-performance Computing (NCHC) and the NIH HPC Biowulf cluster for providing computational and storage resources. This research was done when Chien-Yu Chiou was visiting the National Institute of Neurological Disorders and Stroke, National Institutes of Health, Bethesda, MD, USA.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, A. Singh and J. Zhu, Eds., Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [3] W. Huang, M. Ye, Z. Shi, H. Li, and B. Du, "Rethinking federated learning with domain shift: A prototype view," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 16312–16322.
- [4] Y. Chen, W. Huang, and M. Ye, "Fair federated learning under domain skew with local consistency and domain diversity," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 12077–12086.
- [5] T. Zhou, Y. Yuan, B. Wang, and E. Konukoglu, "Federated feature augmentation and alignment," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 11119–11135, Dec. 2024.
- [6] Z. Wang, Z. Wang, Z. Wang, X. Fan, and C. Wang, "Federated learning with domain shift eraser," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 4978–4987.
- [7] Z. Wang, X. Fan, J. Qi, C. Wen, C. Wang, and R. Yu, "Federated learning with fair averaging," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1615–1623.
- [8] J. Lim and Y. Kim, "Cross-domain semantic segmentation on inconsistent taxonomy using VLMs," in *Proc. Euro. Conf. Comput. Vis.*, 2024, pp. 18–35.
- [9] Z. Tan, X. Yang, and K. Huang, "Rethinking multi-domain generalization with a general learning objective," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 23512–23522.
- [10] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, "Barlow twins: Self-supervised learning via redundancy reduction," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12310–12320.
- [11] Y. Tan et al., "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. 36th AAAI Conf. Artif. Intell.*, 2022, pp. 8432–8440.
- [12] L. Wang, J. Bian, L. Zhang, C. Chen, and J. Xu, "Taming cross-domain representation variance in federated prototype learning with heterogeneous data domains," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 88348–88372.
- [13] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "FedTGP: Trainable global prototypes with adaptive-margin-enhanced contrastive learning for data and model heterogeneity in federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 15, pp. 16768–16776.
- [14] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2018, pp. 429–450.
- [15] S. P. Karimireddy et al., "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5132–5143.
- [16] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 10708–10717.
- [17] S. J. Reddi et al., "Adaptive federated optimization," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–12.

- [18] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–12.
- [19] K. Luo, X. Li, Y. Lan, and M. Gao, "GradMA: A gradient-memory-based accelerated federated learning with alleviated catastrophic forgetting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 3708–3717.
- [20] H. M. Son, M.-H. Kim, T.-M. Chung, C. Huang, and X. Liu, "FedUV: Uniformity and variance for heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 5863–5872.
- [21] Y. Wei and Y. Han, "Multi-source collaborative gradient discrepancy minimization for federated domain generalization," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 14, pp. 15805–15813.
- [22] M. Mohri, G. Sivek, and A. T. Suresh, "Agnostic federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4615–4625.
- [23] T. Li, M. Sanjabi, A. Beirami, and V. Smith, "Fair resource allocation in federated learning," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–13.
- [24] S. M. Hosseini, M. Sikaroudi, M. Babaie, and H. R. Tizhoosh, "Proportionally fair hospital collaborations in federated learning of histopathology images," *IEEE Trans. Med. Imag.*, vol. 42, no. 7, pp. 1982–1995, Jul. 2023.
- [25] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19767–19788.
- [26] C. Shi, J. Li, H. Zhao, D. Guo, and Y. Chang, "FedLWS: Federated learning with adaptive layer-wise weight shrinking," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–13.
- [27] C. Shi, H. Zhao, B. Zhang, M. Zhou, D. Guo, and Y. Chang, "FedAWA: Adaptive optimization of aggregation weights in federated learning using client vectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 30651–30660.
- [28] S. Farhadkhani, R. Guerraoui, N. Gupta, R. Pinot, and J. J. Stephan, "Byzantine machine learning made easy by resilient averaging of momentums," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 6246–6283.
- [29] Y. Allouah, R. Guerraoui, N. Gupta, A. Jellouli, G. Rizk, and J. J. Stephan, "Adaptive gradient clipping for robust federated learning," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–14.
- [30] Q. Liu, C. Chen, J. Qin, Q. Dou, and P.-A. Heng, "FedDG: Federated domain generalization on medical image segmentation via episodic learning in continuous frequency space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 1013–1023.
- [31] Y. Liu, S. Guo, J. Zhang, Y. Zhan, Q. Zhou, and Y. Wang, "Feature correlation-guided knowledge transfer for federated self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 10544–10557, Jun. 2025.
- [32] M. Hong, J. Yun, I. Jeon, and G. Kim, "FedAvP: Augment local data via shared policy in federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 18090–18121.
- [33] H. Yuan, B. Hui, Y. Yang, P. Burlina, N. Z. Gong, and Y. Cao, "Addressing heterogeneity in federated learning via distributional transformation," in *Proc. Euro. Conf. Comput. Vis.*, 2022, pp. 179–195.
- [34] B. Yang, P. C. Yuen, Y. Zhang, and A. Zeng, "Allosteric feature collaboration for model-heterogeneous federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 3042–3056, Feb. 2025.
- [35] H. Chen, T. Zhu, W. Zhou, and W. Zhao, "AFed: Algorithmic fair federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 8, pp. 1–13, Aug. 2025.
- [36] Y. Yan et al., "A simple data augmentation for feature distribution skewed federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 25749–25758.
- [37] Y. Yan, X. Tong, and S. Wang, "Clustered federated learning in heterogeneous environment," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 12796–12809, Sep. 2024.
- [38] H. Huang et al., "Active client selection for clustered federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 16424–16438, Nov. 2024.
- [39] Y. Guo, X. Tang, and T. Lin, "Enhancing clustered federated learning: Integration of strategies and improved methodologies," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–14.
- [40] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6357–6368.
- [41] X. Yang, W. Huang, and M. Ye, "FedAS: Bridging inconsistency in personalized federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 11986–11995.
- [42] H. Li et al., "FedTP: Federated learning by transformer personalization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13426–13440, Oct. 2024.
- [43] L. Meng et al., "Improving global generalization and local personalization for federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 76–87, Jan. 2025.
- [44] Y. Zhang et al., "Federated discriminative representation learning for image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 3204–3217, Feb. 2025.
- [45] J. Luo, C. Chen, and S. Wu, "Mixture of experts made personalized: Federated prompt learning for vision-language models," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–14.
- [46] X. Liu, Y. Deng, A. Nallanathan, and M. Bennis, "Federated learning and meta learning: Approaches, applications, and directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 571–618, 1st Quart., 2024.
- [47] S. Liu et al., "AFM3D: An asynchronous federated meta-learning framework for driver distraction detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 9659–9674, Aug. 2024.
- [48] Q. Wang, S. Xu, R. Xu, and B. Ai, "A model-prediction-based hierarchical personalized federated learning framework with distributed resource optimization," *IEEE Trans. Commun.*, vol. 73, no. 12, pp. 14212–14226, Dec. 2025.
- [49] L. You, Z. Guo, C. Yuen, C. Y.-C. Chen, Y. Zhang, and H. V. Poor, "A framework reforming personalized Internet of Things by federated meta-learning," *Nature Commun.*, vol. 16, no. 1, p. 3739, Apr. 2025.
- [50] Y. Li, W. Xu, H. Wang, Y. Qi, J. Guo, and R. Li, "Personalized federated domain-incremental learning based on adaptive knowledge matching," in *Proc. Euro. Conf. Comput. Vis.*, 2024, pp. 127–144.
- [51] Y. Zhang, H. Zhu, A. Z. Tan, D. Yu, L. Huang, and H. Yu, "PFedMxM: Personalized federated class-incremental learning with mixture of frequency aggregation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 30640–30650.
- [52] Y. Li, H. Wang, Y. Qi, W. Liu, and R. Li, "Re-Fed+: A better replay strategy for federated incremental learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 7, pp. 5489–5500, Jul. 2025.
- [53] Y. Dai, Z. Chen, J. Li, S. Heinecke, L. Sun, and R. Xu, "Tackling data heterogeneity in federated learning with class prototypes," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 6, pp. 7314–7322.
- [54] J. Shi, S. Zheng, X. Yin, Y. Lu, Y. Xie, and Y. Qu, "CLIP-guided federated learning on heterogeneity and long-tailed data," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 13, pp. 14955–14963.
- [55] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, vol. 139, 2021, pp. 8748–8763.
- [56] X. Mu et al., "FedProc: Prototypical contrastive federated learning on non-IID data," *Future Gener. Comput. Syst.*, vol. 143, pp. 93–104, Jun. 2023.
- [57] G. Wan, W. Huang, and M. Ye, "Federated graph learning under domain shift with generalizable prototypes," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 14, pp. 15429–15437.
- [58] J. Han, C. Shan, and Y. Han, "FedGMKD: An efficient prototype federated learning framework through knowledge distillation and discrepancy-aware aggregation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 118326–118356.
- [59] L. Fu, S. Huang, Y. Lai, T. Liao, C. Zhang, and C. Chen, "Beyond federated prototype learning: Learnable semantic anchors with hyper-spherical contrast for domain-skewed data," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 16, pp. 16648–16656.
- [60] Y. Zhou et al., "FedSA: A unified representation learning via semantic anchors for prototype-based federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 21, pp. 23009–23017.
- [61] S. Guo, H. Wang, S. Lin, Z. Kou, and X. Geng, "Addressing skewed heterogeneity via federated prototype rectification with personalization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 5, pp. 8442–8454, May 2025.
- [62] X. An et al., "Federated learning with only positive labels by exploring label correlations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 4, pp. 7651–7665, Apr. 2025.
- [63] A. Arunan, Y. Qin, X. Li, and C. Yuen, "Intelligently augmented contrastive tensor factorization: Empowering multi-dimensional time series classification in low-data environments," *Expert Syst. Appl.*, vol. 287, Aug. 2025, Art. no. 127889.
- [64] W. Yang et al., "FedMPS: Federated learning in a synergy of multi-level prototype-based contrastive learning and soft label generation," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–14, 2025.

- [65] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [66] C.-Y. Chiou, K.-T. Lee, C.-R. Huang, and P.-C. Chung, "ADMM-SRNet: Alternating direction method of multipliers based sparse representation network for one-class classification," *IEEE Trans. Image Process.*, vol. 32, pp. 2843–2856, 2023.
- [67] Y.-T. Chen, C.-Y. Chiou, and C.-R. Huang, "Interactive augmentations, features, and parameters for contrastive learning [AI-eXplained]," *IEEE Comput. Intell. Mag.*, vol. 19, no. 1, pp. 79–80, Feb. 2024.
- [68] S. Sarfraz, V. Sharma, and R. Stiefelhagen, "Efficient parameter-free clustering using first neighbor relations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8934–8943.
- [69] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [70] J. J. Hull, "A database for handwritten text recognition research," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 5, pp. 550–554, May 1994.
- [71] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *Proc. NIPS Workshop Deep Learn. Unsupervised Feature Learn.*, 2024, p. 4.
- [72] P. Roy, S. Ghosh, S. Bhattacharya, and U. Pal, "Effects of degradations on deep neural network architectures," 2018, *arXiv:1807.10108*.
- [73] G. Griffin, A. Holub, and P. Perona, "Caltech-256 object category dataset," California Institute Technology Pasadena, Pasadena, CA, USA, Tech. Rep. 7694, 2007.
- [74] K. Saenko, B. Kulis, M. Fritz, and T. Darrell, "Adapting visual category models to new domains," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2010, pp. 213–226.
- [75] L. Van der Maaten and G. E. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.



Chien-Yu Chiou (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, in 2014, 2016, and 2025, respectively.

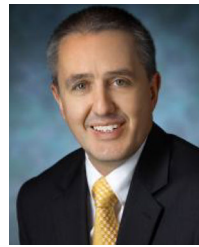
He is currently a Post-Doctoral Researcher with the Department of Electrical Engineering, National Cheng Kung University. His research interests include computer vision, image processing, deep representation learning, and medical image processing.



Chun-Rong Huang (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, in 1999 and 2005, respectively.

In 2005, he joined the Institute of Information Science, Academia Sinica, Taipei, Taiwan, as a Post-Doctoral Fellow. He joined the Department of Computer Science and Engineering, National Chung Hsing University, Taichung, Taiwan, in 2010, where he became a Full Professor in 2019. In 2023, he joined the Cross College Elite Program and the Academy of Innovative Semiconductor and Sustainable Manufacturing, National Cheng Kung University. In 2024, he joined the Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan. His research interests include computer vision, multimedia signal processing, image processing, and medical image processing.

Dr. Huang is a member of the IEEE Circuits and Systems Society, the IEEE Signal Processing Society, the IEEE Computational Intelligence Society, the IEEE Engineering Medicine and Biology Society, and the Phi Tau Phi Honor Society.



Lawrence L. Latour received the Ph.D. degree in biomedical engineering from Worcester Polytechnic Institute, Worcester, MA, USA, in 1994.

He is currently a Senior Scientist with the National Institutes of Neurological Disorders and Stroke, Bethesda, MD, USA, and the PI of the Acute Cerebrovascular Diagnostics Unit. After completing a US DOE Alexander Hollander Distinguished Post-Doctoral Fellowship at Brookhaven National Laboratory, he became the Director of Applied Science at Medical Advances with an appointment in Medical Physics at the Medical College of Wisconsin, Milwaukee, WI, USA. He was recruited to NINDS-DIR as a Staff Scientist in 2000 to help build the Section on Stroke Diagnostics and Therapeutics to conduct clinical research at the Stroke Centers of Suburban Hospital, Bethesda, MA, USA, and Medstar Washington Hospital Center, Washington, DC, USA. He assembled a team to study civilians with acute the Head injury as part of the DOD-funded Center for Neuroscience and Regenerative Medicine. His research interests include the use of MRI to better identify, classify, and treat acute stroke and traumatic brain injury.



Yang C. Fann is currently the Director of Clinical Informatics for the Intramural Research Program (IRP), National Institutes of Health (NIH), Bethesda, MD, USA. In this role, he leads the development of an interoperable clinical data ecosystem, including enterprise clinical systems and data warehouse infrastructure, to support IRP-wide clinical research operations, streamline collaboration, simplify compliance, and enable secure data sharing across diverse clinical information systems. His research spans computational biology, bioinformatics, and clinical research informatics, with a strong focus on applying machine learning and artificial intelligence to translational biomedical research. His recent work includes developing deep learning models for medical image analysis, such as coronary vessel segmentation and angiography, and predictive analytics for stroke outcomes using nationwide population registries. His broader research contributions encompass natural language processing of clinical reports, cloud-based genomics workflows for SARS-CoV-2 analysis, federated learning approaches for digital pathology, and transcriptomic studies in stem cell research and cancer biology.



Pau-Choo Chung (Life Fellow, IEEE) is currently a Distinguished Professor with National Cheng Kung University (NCKU), Tainan, Taiwan, and an Adjunct Professor with National Chung Cheng University, Chiayi, Taiwan. She served as the Program Director of the Intelligent Computing Division, Ministry of Science and Technology, the Dean of the Miin Wu School of Computing, and the Dean of the College of Electrical Engineering and Computer Science, NCKU. Her research interests include computational intelligence, machine learning, computational

pathology, medical image analysis, and pattern recognition, with a focus on whole slide image (WSI) pathology image analysis, and she has built the ALOVAS platform, which was certified by TFDA.

Dr. Chung served as an ADCOM member for two terms, the Chair for the CIS Distinguished Lecturer Program and the Women in Computational Intelligence, the VP for Members Activities, and the VP for Education of IEEE CIS. She also served on two terms of the BoG Member of the IEEE Circuit and Systems Society. She is a member of Phi Tau Phi Honor Society. Her research has won her several awards. She also served as an Associate Editor for IEEE TRANSACTIONS ON NEURAL NETWORK AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS, and IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE.